Mental Health Text Analytics API

Technical Report - COMP3011 Web Services and Web Data

1 Project Overview

This report presents the design and implementation of a secure, database-driven RESTful API for detecting mental health indicators (Depression, Anxiety and Normal) from user generated text. The system integrates machine learning with modern web service architecture to provide both prediction capabilities and analytical insights. A full list of Implemented features is provided in Appendix A.

The API enables authenticated users to:

- Create and manage text posts (CRUD operations)
- Automatically receives ML based predictions under Quick Predict/List Predictions/Get Prediction
- Access full prediction history for auditability
- View aggregated analytics and prediction statistics

The overall aim of the system is to demonstrate the integration of machine learning within a scalable web API while following best practices in backend design, security and data management.

1.1 API Endpoints

A total of 15 endpoints were implemented, covering authentication, CRUD operations, prediction services and system health monitoring.

Method	Endpoint	Description	Auth
GET	/health	System liveness check	No
GET	/health/db	Database connectivity check	No
POST	/auth/register	Register a new user account	No
POST	/auth/login	Login and receive JWT token	No
GET	/posts/	List all posts	Yes
POST	/posts/	Create post + auto ML prediction (Create)	Yes
GET	/posts/{post_id}	Retrieve a single post by ID (Read)	Yes
PUT	/posts/{post_id}	Update post text + re-predict (Update)	Yes
DELETE	/posts/{post_id}	Delete post and cascade predictions (Delete)	Yes
GET	/posts/{post_id}/prediction/latest	Get latest prediction for post	Yes
GET	/posts/{post_id}/prediction/history	Full prediction history for post	Yes
GET	/predictions/	List all latest predictions	Yes

Method	Endpoint	Description	Auth
GET	/predictions/{prediction_id}	Get single prediction	Yes
POST	/predict	Direct ML inference endpoint (rate limited)	No
GET	/analytics/summary	Summary of the API analytics	No

2 Technology Stack and Justification

The technology stack was selected through comparative evaluation of alternatives, focusing on performance, scalability, and integration with machine learning workflows.

2.1 Framework - FastAPI (Python)

FastAPI was selected over Django REST Framework and Node.js Express due to:

1. Asynchronous support - improves scalability for concurrent API requests
2. Automatic OpenAPI/Swagger UI documentation - reduces manual documentation overhead
3. Pydantic validation - ensures strict schema enforcement between client input and application logic
4. Seamless ML integration - python was the natural language choice given the ML pipeline dependency on scikit-learn.

FastAPI also enables a clean separation between ML and API layers while maintaining low latency inference, which is critical in real-time prediction systems

2.2 Database - PostgreSQL with SQLAlchemy ORM

PostgreSQL was chosen over MySQL due to its advanced querying capabilities (e.g., DISTINCT ON), used in the prediction endpoint to efficiently retrieve the latest prediction per post and its superior handling of time zone-aware timestamps. This was required for accurate audit trial ordering. It also supports **future analytical extensions**, such as cohort analysis or temporal prediction trends - aligning the system with real-world healthcare analytics use cases

SQLAlchemy ORM enforces a clean separation between the data and service layers; models are defined once in python, and the database schema is derived from them. Alembic manages schema versioning through migration scripts, enabling reproducible database initialisation on any environment including the deployment server via alembic upgrade head.

SQLite was employed in the test environment to eliminate the requirement for running database instances during automated testing.

2.3 JWT Authentication

JWT-based authentication was selected to maintain a **stateless architecture**, consistent with REST principles.

- Each token encodes the user's identity and is signed using HS256, allowing any request to be validated without a database lookup.
- Passwords hashed using bcrypt (via passlib library), ensuring plaintext credentials are never persisted.
- Token expiry configurable via environmental variables (default: 60 minutes).

Stateless authentication ensures scalability and avoids server-side session storage, making the API suitable for distributed deployment.

2.4 Machine Learning Model (Logistic Regression + TF-IDF)

A **scikit-learn pipeline** combining TF-IDF vectorisation and Logistic Regression was implemented.

Reasons for selection:

- Fast inference time (suitable for APIs)
- Interpretability (important for mental health context)
- Low computational overhead

The model represents a **deliberate trade-off between accuracy and performance**, prioritising real-time usability over state-of-the-art performance

3 System Architecture and Design

The system follows a layered service-oriented architecture, ensuring modularity and maintainability.

Layer	Location	Responsibility
API Layer	app/api/	Route definitions, request/response handling
Schema Layer	app/schemas/	Pydantic models for input validation and serialisation
Service Layer	app/services/	Business logic - ML inference via predictor.py
Model Layer	app/models/	SQLAlchemy ORM models (User, Post, Prediction)
Database Layer	app/db/	Session management, connection pooling via Alembic

3.1 Design Decision: ML Integration and Dataset Pipeline

The training dataset was sourced from Kaggle **(Sarkar, 2025) –** and consists of text samples labelled with mental health categories. A dedicated script (train_model.py) was developed to handle data processing, model training, and evaluation, forming a complete and reproducible machine learning pipeline.

Pipeline steps:

1. Data Validation to ensure consistency of text and labels
2. Stratified 80/20 train-test split to preserve class distribution
3. TF-IDF vectorisation (bigrams + stopwords) for feature extraction
4. Logistic Regression training for classification
5. Model evaluation (precision, recall, F1-score)
6. Serialisation using joblib

The pipeline ensures reproducibility, a key requirement in both ML and software engineering.

3.2 Design Decision: Prediction Audit Trail

A critical design feature is the text_snapshot field stored with every prediction.

- Each prediction stores the exact input text used
- New predictions are created on updates (no overwriting)
- Enables full historical traceability

This transforms the system from a simple classifier into an auditable analytical system, which is essential in sensitive domains such as mental health.

3.3 Design Decision: Analytics Endpoint

A dedicated endpoint /analytics/summary was implemented to provide aggregated statistics including total posts, total predictions, label counts and percentage distribution.

Although similar metrics are displayed in the frontend dashboard, exposing this as a backend endpoint improves:

- separation of concerns
- reusability across clients
- scalability for future features

This elevates the API from a simple CRUD system to a data-driven analytical service.

4 Testing Approach

A comprehensive **test suite (49 tests)** was implemented using pytest.

Key strategies:

- SQLite in-memory database for isolation
- Mocking ML model to avoid dependency on model file
- Coverage across authentication, CRUD, predictions, and edge cases

Testing focuses on **system behaviour rather than implementation**, ensuring robustness under real-world scenarios

Test Class	Coverage
TestHealthEndpoints	API liveness and database connectivity
TestAuthRegister	Registration, duplicate emails, validation errors
TestAuthLogin	Login success, wrong credentials, missing fields
TestJWTProtection	Protected routes reject missing/invalid tokens
TestPostCreate	CRUD create, auto-prediction trigger, missing fields
TestPostRead	Get by ID, list all, 404 handling
TestPostUpdate	Text update triggers new prediction, source-only does not
TestPostDelete	Delete removes post and cascades predictions
TestPredictions	Latest, history ordering, text snapshot integrity
TestDirectPredict	Fields, confidence range, rate limit (429), DB persistence

5 Challenges and Lessons Learned

5.1 SQLite and PostgreSQL Timestamp Precision

Differences between SQLite (used for testing) and PostgreSQL (used in production) led to inconsistencies in timestamp precision. This resulted in unreliable ordering of prediction records during testing. Resolution: Test assertions were adapted to avoid strict ordering assumptions and instead validate expected data integrity.

5.2 Authentication Behaviour

FastAPI returns 403 (Forbidden) for missing credentials instead of more commonly expected 401 (Unauthorized). This required careful handling in test cases and highlighted the importance of understanding framework-specific behaviour.

5.3 Data integrity issues

Initial omission of required fields such as text_snapshot caused database errors during prediction creation. This emphasised the importance of enforcing strict schema validation and ensuring consistency between application logic and database constraints.

5.4 Deployment Challenges

PythonAnywhere disk quota limitation - Deployment failed due to the 512MB storage limit on the free tier. Machine learning dependencies (NumPy, SciPy, scikit-learn) exceeded the quota, required moving to an alternative web hosting platform (Render).

Local vs production environment mismatch - Database configurations using “localhost” functioned locally but failed in deployment, as remote servers cannot access local machines resources. This required setting up a cloud-hosted PostgreSQL database and managing separate environment configuration.

Limited debugging visibility in deployment - Unlike local development, Render provided limited logs, making it difficult to diagnose HTTP 500 errors encountered during authentication (login or registration). This required manual inspection of deployment logs and iterative debugging.

Key Lessons Learned: A system functioning locally may fail in production due to differences in environment, configuration and infrastructure. Proper handling of environment variables and dependencies is critical.

6 Limitations and Future Improvements

ML Model Accuracy and Nuanced Language - The Logistic Regression model (currently used) classifies short, informal text with low confidence. For example, the phrase 'I am not feeling okay' was classified as Normal with 33% confidence, below the uncertainty threshold. This is a fundamental limitation of bag-of-words approaches, which cannot capture semantic context, negation, or sentiment nuance. A fine-tuned transformer model such as DistilBERT would substantially improve classification quality at the cost of increased inference latency and deployment complexity.

Absence of Single-User Data Isolation - Currently, any authenticated user can retrieve all posts regardless of authorship. A production implementation would enforce user_id filtering on all list endpoints, exposing only records belonging to the requesting user. This would require a foreign key relationship between Post and User and modifications to the query logic across the posts router.

Frontend State Management - The frontend dashboard is implemented using plain JavaScript with global state variables. For a larger application this approach becomes difficult to maintain. A component-based framework such as React or Vue, combined with a formal state management library, would improve maintainability and enable more sophisticated UI interactions.

7 Generative AI Declaration

Generative AI tools (including ChatGPT and Claude) were used to assist in:

- Evaluating alternative technology stacks and architectural approaches, including the adoption of a service-layer design pattern and the use of audit fields such as text_snapshot.
- Assisting in the design and integration of the machine learning pipeline following data extraction and preprocessing from the Kaggle dataset (Generation of a subset of Kaggle dataset with 500 lines).
- Code debugging and troubleshooting, including the test suite was developed collaboratively with Claude.
- Assisting in frontend development, including the implementation of an HTML/CSS/JavaScript dashboard with a modular structure (api.js, auth.js, dashboard.js, posts.js, predictions.js).
- Improving structure, clarity and academic quality of the technical report
- Suggesting improvements in API design and deployment strategies

All AI generated outputs were critically reviewed, validated, and adapted by the author to ensure correctness and alignment with coursework requirements. No content was used without verification, and the final implementation reflects the author’s own understanding and development work.

References

1. Sarkar, S. 2025. Sentiment Analysis for Mental Health. Kaggle. [Online]. [Accessed 27 February 2026]. Available at: https://www.kaggle.com/datasets/suchintikasarkar/sentiment-analysis for-mental-health

Appendix A: Key Features

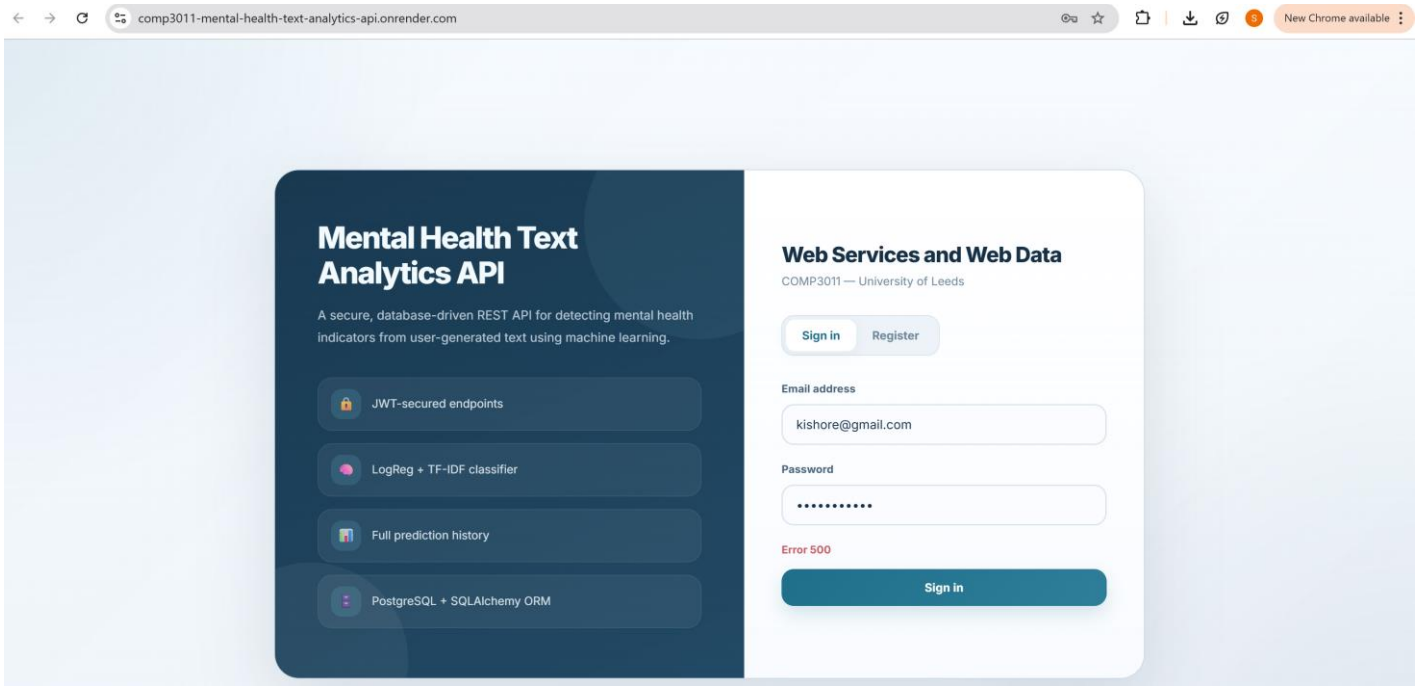
1. **JWT-secured endpoints** - all post and prediction routes require a valid Bearer token; unprotected routes return 401/403 appropriately.
2. **Automatic ML prediction on post creation** - every new post triggers an inference call, eliminating the need for a separate prediction step.
3. **Re-prediction on text update with history tracking** - when text in post is changed or updated, a new prediction row is created while the old one is preserved to maintain full history of the post.
4. **Cascade deletion for data consistency** - deleting a post removes all associated predictions via SQLAlchemy cascade, preventing orphaned rows.
5. **Confidence-based uncertainty flagging** - the /predict endpoint returns an uncertain: true flag when model confidence falls below 40% enabling downstream handling of low-confidence results.
6. **Rate limiting (SlowAPI)** - the /predict endpoint is rate-limited to 10 requests per minute per IP using SlowAPI, returning HTTP 429 on excess, protecting against abuse
7. **Model versioning** - each prediction stores the model_version string, enabling future model upgrades without losing historical comparison data.
8. **Frontend Dashboard** - a multi-page HTML/CSS/JS frontend served by FastAPI itself, providing login, CRUD, prediction visualisation, and a donut chart showing label distribution.
9. **Health Monitoring endpoints** - /health and /health/db provide operational monitoring with no authentication required.

Appendix B: Deployment

URL of the Deployed Site: https://comp3011-mental-health-text-analytics-api.onrender.com

The application has been successfully deployed and is live on Render. However, the deployed instance currently encounters an HTTP 500 (Internal Server Error), which prevents users from logging into the system. Despite multiple debugging attempts, this issue could not be fully resolved within the project timeframe.

As a result, the system will be demonstrated via local execution during the presentation, where full functionality, including user authentication, post management, and machine learning predictions - can be showcased**.**



Appendix C: Snippets of Some Questions Asked to Gen AI

Use of Logistic Regression as the ML model

Question

Should I use Logistic Regression or BERT for detecting depression and anxiety in social media text?

Answer

Both models have advantages depending on the project scope:

- **Logistic Regression (with TF-IDF)**
  - Fast and lightweight
  - Easy to integrate into APIs
  - Works well for baseline models
  - Suitable for coursework constraints
- **BERT (Deep Learning)**
  - Captures context better
  - Higher accuracy for nuanced language
  - Requires more computation and setup

For this project, a good approach is:

- Use **Logistic Regression as a baseline**
- Optionally compare with **BERT for evaluation**

This demonstrates:

- Understanding of traditional vs deep learning approaches
- Strong experimental design

API Design Help

Question

What endpoints should I include in a REST API for a mental health prediction system?

Answer

A well-structured API should include:

Authentication

- `POST /register`
- `POST /login`

Posts

- `POST /posts`
- `GET /posts`
- `PUT /posts/{id}`
- `DELETE /posts/{id}`

Predictions

- `POST /predict/{post_id}`
- `GET /predictions`
- `GET /predictions/{id}`

This ensures:

- Full CRUD functionality
- Clear separation between data and ML operations
- RESTful design principles

Evaluation Metrices Choice

Question

What evaluation metrics should I use for a mental health text classification model?

Answer

Key metrics include:

- **Accuracy** – overall correctness
- **Precision** – how many predicted positives are correct
- **Recall** – ability to detect actual positives
- **F1-score** – balance between precision and recall

For mental health detection:

- **Recall is especially important**
  (missing a depressed case is more critical than false positives)

Database Design Choices

Question

How should I design my database to store predictions for audit and reproducibility?

Answer

A good design is to include a **prediction history table** with the following fields:

- `id` – primary key
- `post_id` – reference to the original post
- `prediction_label` – Depression / Anxiety / Normal
- `confidence_score` – probability output
- `text_snapshot` – copy of the text at prediction time
- `created_at` – timestamp

The **text_snapshot field is critical** because:

- Posts may change over time
- Predictions must remain reproducible
- Supports auditing and traceability

This design aligns with best practices in **ML system accountability**.